

```
#!/usr/bin/env python3
```

```
"""
```

```
sqlite_mcp.py
```

```
-----
```

Lightweight Model Context Protocol (MCP) server exposing local SQLite databases to cognitive agent swarms (OpenWiki TUI, Prime Agent, ECC, Claude Code CLI).

Supports:

- tools/list: list_tables, describe_table, execute_read_query, execute_write_query
- tools/call: safe execution against local databases (e.g. audit_ledger.db, memory caches)
- Stdio JSON-RPC 2.0 transport

```
"""
```

```
import sys
```

```
import os
```

```
import json
```

```
import sqlite3
```

```
from typing import Any, Dict, List
```

```
DEFAULT_DB_PATH = os.environ.get(
    "SQLITE_MCP_DB_PATH",
    os.path.expanduser("~/novae-xorpus/data_vault/sqlite_memory.db")
)
```

```
def get_connection(db_path: str = DEFAULT_DB_PATH) -> sqlite3.Connection:
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row
    return conn
```

```
def list_tables(db_path: str = DEFAULT_DB_PATH) -> List[str]:
    with get_connection(db_path) as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT name FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%';")
        return [row[0] for row in cursor.fetchall()]
```

```
def describe_table(table_name: str, db_path: str = DEFAULT_DB_PATH) -> List[Dict[str, Any]]:
    with get_connection(db_path) as conn:
        cursor = conn.cursor()
        cursor.execute(f"PRAGMA table_info({table_name});")
        columns = []
        for row in cursor.fetchall():
            columns.append({
```

```

        "cid": row[0],
        "name": row[1],
        "type": row[2],
        "notnull": bool(row[3]),
        "default_value": row[4],
        "pk": bool(row[5])
    })
    return columns

```

```

def execute_read_query(query: str, params: list = None, db_path: str = DEFAULT_DB_PATH) ->
List[Dict[str, Any]]:
    if params is None:
        params = []
    normalized = query.strip().upper()
    if not (normalized.startswith("SELECT") or normalized.startswith("PRAGMA") or
normalized.startswith("EXPLAIN")):
        raise ValueError("Only read-only queries (SELECT, PRAGMA, EXPLAIN) are permitted in
execute_read_query.")

```

```

    with get_connection(db_path) as conn:
        cursor = conn.cursor()
        cursor.execute(query, params)
        rows = cursor.fetchall()
        return [dict(row) for row in rows]

```

```

def execute_write_query(query: str, params: list = None, db_path: str = DEFAULT_DB_PATH) ->
Dict[str, Any]:
    if params is None:
        params = []
    with get_connection(db_path) as conn:
        cursor = conn.cursor()
        cursor.execute(query, params)
        conn.commit()
        return {
            "rows_affected": cursor.rowcount,
            "last_row_id": cursor.lastrowid
        }

```

```

def get_tool_definitions() -> List[Dict[str, Any]]:
    return [
        {
            "name": "sqlite_list_tables",
            "description": "Lists all non-system tables in the target SQLite database.",

```

```

    "inputSchema": {
      "type": "object",
      "properties": {
        "db_path": {"type": "string", "description": "Optional path to SQLite database file."}
      }
    },
    {
      "name": "sqlite_describe_table",
      "description": "Returns column metadata and constraints for a specified SQLite table.",
      "inputSchema": {
        "type": "object",
        "required": ["table_name"],
        "properties": {
          "table_name": {"type": "string"},
          "db_path": {"type": "string"}
        }
      }
    },
    {
      "name": "sqlite_read_query",
      "description": "Executes a read-only SELECT query against the SQLite database.",
      "inputSchema": {
        "type": "object",
        "required": ["query"],
        "properties": {
          "query": {"type": "string"},
          "params": {"type": "array", "items": {"type": "string"}},
          "db_path": {"type": "string"}
        }
      }
    },
    {
      "name": "sqlite_write_query",
      "description": "Executes an INSERT, UPDATE, or DELETE query against the SQLite
database.",
      "inputSchema": {
        "type": "object",
        "required": ["query"],
        "properties": {
          "query": {"type": "string"},
          "params": {"type": "array", "items": {"type": "string"}},
          "db_path": {"type": "string"}
        }
      }
    }
  ]
}

```

```

    }
  }
}
]

```

```

def main():
    while True:
        line = sys.stdin.readline()
        if not line:
            break
        try:
            req = json.loads(line)
            req_id = req.get("id")
            method = req.get("method")
            params = req.get("params", {})

            if method == "initialize":
                response = {
                    "jsonrpc": "2.0",
                    "id": req_id,
                    "result": {
                        "protocolVersion": "2024-11-05",
                        "serverInfo": {"name": "sqlite-mcp-server", "version": "1.0.0"},
                        "capabilities": {"tools": {}}
                    }
                }
            elif method == "tools/list":
                response = {
                    "jsonrpc": "2.0",
                    "id": req_id,
                    "result": {"tools": get_tool_definitions()}
                }
            elif method == "tools/call":
                name = params.get("name")
                args = params.get("arguments", {})
                db_p = args.get("db_path", DEFAULT_DB_PATH)

                if name == "sqlite_list_tables":
                    res = list_tables(db_p)
                elif name == "sqlite_describe_table":
                    res = describe_table(args["table_name"], db_p)
                elif name == "sqlite_read_query":
                    res = execute_read_query(args["query"], args.get("params", []), db_p)

```

```

elif name == "sqlite_write_query":
    res = execute_write_query(args["query"], args.get("params", []), db_p)
else:
    raise ValueError(f"Unknown tool: {name}")

response = {
    "jsonrpc": "2.0",
    "id": req_id,
    "result": {
        "content": [{"type": "text", "text": json.dumps(res, indent=2)}]
    }
}
else:
    response = {
        "jsonrpc": "2.0",
        "id": req_id,
        "error": {"code": -32601, "message": f"Method {method} not found"}
    }

sys.stdout.write(json.dumps(response) + "\n")
sys.stdout.flush()
except Exception as e:
    err_resp = {
        "jsonrpc": "2.0",
        "id": req.get("id") if 'req' in locals() else None,
        "error": {"code": -32603, "message": str(e)}
    }
    sys.stdout.write(json.dumps(err_resp) + "\n")
    sys.stdout.flush()

if __name__ == "__main__":
    main()

```